Практическая работа 1. Разработка перечня артефактов и протоколов проекта.

Выбранный проект: учебный (Система управления задачами)

Таблица описание проекта:

Параметр	Описание
Название	TaskFlow — система управления задачами для учебных команд
Цель	Предоставить командам инструмент для постановки задач, контроля сроков, отслеживания прогресса и совместной работы над проектами
Целевая аудитория	Студенты, преподаватели, руководители учебных проектных команд, администраторы
Ключевые функции	1. Регистрация и аутентификация пользователей (роли: студент, преподаватель, администратор) 2. Создание и редактирование проектов и задач 2. 3. Назначение исполнителей и установка дедлайнов 3. 4. Канбан-доска с перемещением задач по статусам 4. 5. Комментарии и уведомления к задачам 5. 6. Формирование отчётов о прогрессе (диаграммы, статистика) 6. 7. Экспорт задач и отчётов в CSV/PDF
Участники проекта	Заказчик (преподаватель), аналитик, дизайнер, frontend-разработчик, backend-разработчик, тестировщик, DevOps-инженер, руководитель проекта (PM)

Таблица: Перечень артефактов.

Артефакт	Тип	Назначение	Кто создаёт	Кто использует	Этап жизненного цикла	Формат хранения
Техническое задание	Документ	Фиксирует требования заказчика, границы проекта, критерии приёмки	Аналитик	Вся команда, заказчик	Анализ требований	Word / PDF, Confluence
Спецификация требований (SRS)	Документ	Детальное описание функциональных и нефункциональных требований	Аналитик	Разработчики, тестировщик, заказчик	Анализ требований	PDF, Confluence
Use-case диаграмма	Схема	Визуализация сценариев использования системы акторами	Аналитик	Разработчики, дизайнер, тестировщик	Проектирование	Файл Draw.io / PNG
ER-диаграмма базы данных	Схема	Описание структуры данных, сущностей и связей	Backend-разработчик	Разработчики, тестировщик	Проектирование	Файл dbdiagram.io / PNG
Архитектурная схема системы	Схема	Отображение компонентов системы и их взаимодействия (клиент–сервер–БД)	Backend-разработчик, DevOps	Вся команда	Проектирование	Файл Draw.io / PNG
Макеты интерфейса (UI/UX)	Макет	Прототипы экранов: канбан-доска, форма задачи, отчёты	Дизайнер	Разработчики, заказчик	Проектирование	Figma
Исходный код frontend	Код	Реализация пользовательского интерфейса (React)	Frontend-разработчик	Разработчики, тестировщик	Реализация	Git-репозиторий (GitHub/GitLab)
Исходный код backend	Код	Реализация серверной логики и API (Node.js/Express)	Backend-разработчик	Разработчики, тестировщик	Реализация	Git-репозиторий
Схема базы данных (SQL-скрипты)	Код / конфигурация	Создание таблиц, индексов, миграций	Backend-разработчик	Разработчики, DevOps	Реализация	Git-репозиторий
Конфигурационные файлы	Конфигурация	Настройки окружений (dev/staging/prod), переменные среды	DevOps-инженер	Разработчики, DevOps	Реализация / развёртывание	Git-репозиторий, .env (защищённый
API-документация (Swagger/OpenAPI)	Документ	Описание эндпоинтов, форматов запросов и ответов	Backend-разработчик	Frontend-разработчик, тестировщик	Реализация	Swagger UI, файл YAML
Тест-кейсы	Тест-кейс	Формализованные	Тестировщик	Тестировщик,	Тестирование	Excel / TestRail /

		сценарии проверки функций системы		разработчики		Google Sheets
Отчёт о тестировании	Документ	Результаты прогона тестов, найденные дефекты, статистика	Тестировщик	PM, заказчик, разработчики	Тестирование	PDF / Confluence
Баг-репорты	Документ (запись)	Описание дефектов с шагами воспроизведения	Тестировщик	Разработчики	Тестирование	Трекер (Jira/YouTrack)
Инструкция по развёртыванию	Документ	Пошаговое руководство по установке и запуску системы	DevOps-инженер	DevOps, разработчики	Развёртывание	Markdown в репозитории (README)
Пользовательская документация	Документ	Руководство пользователя: как работать с задачами, отчётами	Аналитик, дизайнер	Пользователи, заказчик	Внедрение / эксплуатация	PDF, Wiki
Протоколы встреч и соглашений	Документ	Фиксация решений, договорённостей и ответственных	PM	Вся команда, заказчик	Все этапы	Confluence / Google Docs
Релизные заметки (Release Notes)	Документ	Описание изменений и новых функций в версии	PM, DevOps	Заказчик, пользователи	Внедрение	Markdown в репозитории, Wiki

Таблица: Перечень протоколов.

Протокол	Назначение	Участники	Периодичность	Форма фиксации
Протокол совещания	Фиксация обсуждаемых вопросов, принятых решений и ответственных за их выполнение	Вся команда, заказчик	Еженедельно	Документ в Confluence / Google Docs
Протокол согласования требований	Регламентирует порядок утверждения и изменения требований заказчиком	Аналитик, заказчик, PM	По событию (при изменении требований)	Документ с подписью / электронное письмо
Протокол код-ревью	Определяет правила проверки кода перед слиянием в основную ветку (checklist, минимум 1 ревьюер	Разработчики, тимлид	По событию (на каждый pull request)	Запись в Git-платформе (GitHub/GitLab)

Протокол тестирования	Регламентирует порядок фиксации результатов тестов, классификацию дефектов и критерии выхода релиза	Тестировщик, разработчики, PM	По событию (перед каждым релизом)	Отчёт в TestRail / документ в Confluence
Протокол приёмки	Определяет процедуру передачи результата заказчику и критерии приёмки	Заказчик, PM, тестировщик	Разово (по завершении этапа/релиза)	Акт приёмки (документ PDF)
Протокол управления изменениями	Регламентирует порядок внесения изменений в требования и оценку их влияния на сроки/бюджет	PM, аналитик, заказчик	По событию	Запись в трекере (Jira) + документ Change Request
Протокол обмена данными между системами	Определяет форматы, частоту и ответственность при интеграции с внешними сервисами (например, email-уведомления	Backend-разработчик, DevOps	Разово (при проектировании интеграции)	Документ в Confluence (спецификация API)

Связь Артефактов и Протоколов:

Пример 1. Протокол согласования требований регулирует, как утверждается артефакт «Техническое задание» и «Спецификация требований (SRS)». Без этого протокола требования могут быть изменены без ведома заказчика, что приведёт к переделке кода и срыву сроков.

Пример 2. Протокол код-ревью напрямую связан с артефактами «Исходный код frontend» и «Исходный код backend». Он определяет, что код попадает в основную ветку только после проверки минимум одним ревьюером. Без этого протокола в репозиторий может попасть некачественный или незадокументированный код, что усложнит поддержку.

Пример 3. Протокол тестирования регулирует создание и использование артефактов «Тест-кейсы», «Отчёт о тестировании» и «Баг-репорты». Он устанавливает, как фиксируются результаты тестов и какие дефекты блокируют релиз. Без этого протокола невозможно объективно оценить готовность продукта к приёмке.

Пример 4. Протокол приёмки связывает артефакты «Отчёт о тестировании», «Релизные заметки» и «Пользовательская документация»: заказчик принимает результат только при наличии полного пакета документов и успешного отчёта о тестировании.

Пример 5. Протокол управления изменениями связан с артефактом «Техническое задание»: любое изменение требований оформляется через Change Request, что гарантирует актуальность ТЗ и прозрачность влияния изменений на проект.

Обоснование выбора:
Обязательные артефакты для любого проекта:

Техническое задание, спецификация требований — без них команда не знает, что делать.

Исходный код — основной результат разработки.

Тест-кейсы и отчёт о тестировании — подтверждают качество.

Инструкция по развёртыванию — без неё систему невозможно запустить.

Протоколы совещаний и согласования требований — обеспечивают управляемость.

Специфичные для нашего проекта:

Макеты интерфейса (Figma) — критичны, так как канбан-доска и отчёты требуют продуманного UX.

API-документация (Swagger) — важна из-за разделения на frontend и backend.

ER-диаграмма и SQL-скрипты — специфичны для системы с реляционной БД (задачи, пользователи, проекты).

Протокол обмена данными между системами — нужен для интеграции email-уведомлений.

Критичные протоколы:

Протокол код-ревью — предотвращает попадание некачественного кода.

Протокол тестирования — гарантирует объективность проверки.

Протокол управления изменениями — защищает от «расползания» требований.

Протокол приёмки — формализует завершение этапа.

Что произойдёт при отсутствии:

Без ТЗ — команда сделает не то, что нужно заказчику.

Без протокола код-ревью — код станет неподдерживаемым, вырастет число багов.

Без тест-кейсов — невозможно системно проверить функциональность, дефекты найдут пользователи.

Без протокола управления изменениями — сроки и бюджет выйдут из-под контроля.

Без инструкции по развёртыванию — система не будет запущена в срок.

Без протокола приёмки — проект не будет формально закрыт, возникнут споры с заказчиком.